

Laboratorio Zero Trust Security con Istio en AKS

- Laboratorio completo: Zero Trust Security con Istio en AKS
 - 1. Objetivo del laboratorio
 - 2. Arquitectura objetivo
 - 3. Distribucion sugerida para 5 horas
 - 4. Prerrequisitos
 - 5. Parte 1 - Validar la aplicacion actual
 - 5.1 Verificar conexion con AKS
 - 5.2 Verificar deployments
 - 5.3 Verificar pods
 - 5.4 Verificar servicios
 - 5.5 Probar acceso actual
 - 6. Parte 2 - Instalar Istio en AKS
 - 6.1 Descargar Istio
 - 6.2 Instalar Istio con perfil demo
 - 7. Parte 3 - Activar sidecar injection
 - 7.1 Etiquetar el namespace de la aplicacion
 - 7.2 Reiniciar deployments
 - 7.3 Validar sidecars
 - 8. Parte 4 - Exponer la aplicacion con Istio Gateway
 - 8.1 Crear Gateway
 - 8.2 Crear VirtualService
 - 8.3 Obtener IP publica del Istio Gateway
 - 9. Parte 5 - Activar mTLS STRICT
 - 9.1 Crear politica mTLS STRICT
 - 9.2 Validar estado de mTLS
 - 10. Parte 6 - Prueba negativa desde fuera del mesh
 - 10.1 Crear namespace sin Istio
 - 10.2 Crear pod de prueba sin sidecar
 - 10.3 Intentar llamada al servicio interno
 - 11. Parte 7 - Crear identidades de servicio
 - 11.1 Crear service accounts
 - 11.2 Asociar service accounts a deployments
 - 12. Parte 8 - AuthorizationPolicy: deny by default
 - 12.1 Crear politica deny-all
 - 12.2 Probar acceso desde Istio Gateway

- 13. Parte 9 - Permitir trafico desde Istio Gateway hacia front
 - 13.1 Identificar service account del ingress gateway
 - 13.2 Crear politica allow desde gateway hacia front
 - 13.3 Crear politica allow desde front hacia back
 - 13.4 Probar acceso nuevamente
- 14. Parte 10 - Validar JWT con Istio
 - 14.1 Instalar dependencias Python
 - 14.2 Crear script para generar JWT y JWKS
 - 14.3 Revisar token valido
- 15. Parte 11 - Crear RequestAuthentication con JWKS embebido
 - 15.1 Preparar JWKS para YAML
 - 15.2 Crear RequestAuthentication
- 16. Parte 12 - Exigir JWT valido y scope correcto
 - 16.1 Eliminar politica anterior
 - 16.2 Crear politica que exige JWT y claim `scope=vote: read`
- 17. Parte 13 - Pruebas de seguridad
 - 17.1 Prueba 1: sin token
 - 17.2 Prueba 2: token valido en firma, pero con scope incorrecto
 - 17.3 Prueba 3: token valido con scope correcto
 - 17.4 Prueba 4: token alterado manualmente
- 18. Parte 14 - Validaciones finales
 - 18.1 Ver politicas de autenticacion
 - 18.2 Ver politicas de autorizacion
 - 18.3 Ver politicas mTLS
 - 18.4 Analizar configuracion de Istio
 - 18.5 Revisar configuracion distribuida al proxy
- 19. Parte 15 - Integracion opcional con Jenkins
 - 19.1 Smoke test sin token
 - 19.2 Smoke test con token valido
- 20. Resultado esperado final
- 21. Troubleshooting
 - Problema 1: Los pods siguen mostrando `1/1`
 - Problema 2: El Gateway no responde
 - Problema 3: Todo responde 403 despues de deny-all
 - Problema 4: El principal del ingress gateway no coincide
 - Problema 5: El JWT siempre falla
 - Problema 6: Istio no reconoce el JWKS embebido
- 22. Limpieza del laboratorio
- 23. Preguntas de cierre para los alumnos
- 24. Mensaje final del laboratorio
- 25. Referencias recomendadas

Laboratorio completo: Zero Trust Security con Istio en AKS

Duracion estimada: 5 horas

Nivel: intermedio-avanzado

Entorno: Azure Kubernetes Service (AKS), Azure Container Registry (ACR), Jenkins, kubectl e Istio

Aplicacion base sugerida: `azure-vote-front` + `azure-vote-back` / Redis

1. Objetivo del laboratorio

En este laboratorio, los participantes tomara una aplicacion ya desplegada en AKS mediante Jenkins y la protegeran con Istio Service Mesh.

Al finalizar, la arquitectura tendra:

```
Cliente
-> Istio Ingress Gateway
-> Validacion JWT con RequestAuthentication
-> AuthorizationPolicy basada en claims
-> azure-vote-front
-> mTLS STRICT
-> AuthorizationPolicy service-to-service
-> azure-vote-back / Redis
```

Durante el laboratorio se implementara:

- Instalacion de Istio sobre AKS.
 - Inyeccion de sidecar Envoy en los workloads.
 - Publicacion de la aplicacion mediante Istio Gateway y VirtualService.
 - Comunicacion interna protegida con mTLS en modo STRICT.
 - Microsegmentacion con AuthorizationPolicy.
 - Validacion de JWT desde Istio con RequestAuthentication.
 - Autorizacion basada en claims del token.
 - Pruebas positivas y negativas de seguridad.
 - Integracion opcional con Jenkins como Security as Code.
-

2. Arquitectura objetivo

```
GitHub
-> Jenkins
-> Docker build
-> ACR
-> AKS
-> Istio Ingress Gateway
-> Envoy sidecar
-> Microservicios protegidos
```

El objetivo no es solo desplegar una aplicacion, sino convertirla en una arquitectura Zero Trust:

- Nadie es confiable por defecto.
- Todo trafico debe estar autenticado.
- Todo acceso debe estar autorizado.
- La comunicacion interna debe estar cifrada.
- Las politicas deben ser declarativas y versionables.

3. Distribucion sugerida para 5 horas

Bloque	Actividad	Tiempo estimado
1	Validar AKS y aplicacion existente	30 min
2	Instalar Istio y activar sidecars	45 min
3	Exponer la app con Istio Gateway	40 min
4	Activar mTLS STRICT	45 min
5	Crear identidades y AuthorizationPolicy	60 min
6	Validar JWT con Istio	60 min
7	Pruebas, troubleshooting y debrief	40 min

4. Prerrequisitos

Antes de iniciar, se asume que ya existe un entorno con:

- AKS funcionando.
- `kubectl` conectado al cluster.
- Jenkins desplegando hacia AKS.
- Azure Container Registry operativo.
- Aplicacion `azure-vote` desplegada.
- Deployments similares a:
 - `azure-vote-front`
 - `azure-vote-back`
- Servicios Kubernetes similares a:
 - `azure-vote-front`
 - `azure-vote-back`

Herramientas necesarias:

```
az version
kubectl version --client
helm version
python3 --version
```

Si se trabaja desde Azure Cloud Shell, normalmente `az`, `kubectl` y `helm` ya estan disponibles.

5. Parte 1 - Validar la aplicacion actual

5.1 Verificar conexion con AKS

```
kubectl get nodes
```

Resultado esperado:

NAME	STATUS	ROLES	AGE	VERSION
aks-nodepool1-xxxxxxx-vmss000000	Ready	agent	...	v1.xx.x

5.2 Verificar deployments

```
kubectl get deploy
```

Resultado esperado:

NAME	READY	UP-T0-DATE	AVAILABLE	AGE
azure-vote-back	1/1	1	1	...
azure-vote-front	1/1	1	1	...

5.3 Verificar pods

```
kubectl get pods
```

Resultado esperado antes de Istio:

NAME	READY	STATUS	RESTARTS	AGE
azure-vote-back-xxxxxxxx-xxxx	1/1	Running	0	...
azure-vote-front-xxxxxxxx-xxxx	1/1	Running	0	...

Observacion: antes de habilitar Istio, normalmente cada pod tiene 1/1 contenedores. Despues de inyectar Envoy, deberia tener 2/2.

5.4 Verificar servicios

```
kubectl get svc
```

Resultado esperado:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
azure-vote-back	ClusterIP	...	<none>	6379/TCP
azure-vote-front	LoadBalancer	...	x.x.x.x	80:xxxxx/TCP

5.5 Probar acceso actual

```
export APP_IP=<EXTERNAL-IP-ACTUAL>
curl http://$APP_IP
```

Si la aplicacion responde, el punto de partida esta listo.

Mensaje para explicar en clase:

En este momento la aplicacion funciona, pero todavia no tenemos un modelo fuerte de Zero Trust. La comunicacion interna puede estar abierta, no tenemos control declarativo de quien puede hablar con quien, y la validacion de tokens depende de la aplicacion o de algun gateway externo.

6. Parte 2 - Instalar Istio en AKS

Para este laboratorio se usara `istioctl`, porque permite una instalacion simple, portable y facil de demostrar. En un entorno productivo de AKS tambien podria evaluarse el add-on administrado de Istio.

6.1 Descargar Istio

```
curl -L https://istio.io/downloadIstio | sh -
```

Entrar al directorio descargado:

```
cd istio-*
export PATH=$PWD/bin:$PATH
```

Validar:

```
istioctl version
```

6.2 Instalar Istio con perfil demo

```
istioctl install --set profile=demo -y
```

Validar pods de Istio:

```
kubectl get pods -n istio-system
```

Resultado esperado:

NAME	READY	STATUS	RESTARTS	AGE
istio-ingressgateway-xxxxxxxxxx-xxxxx	1/1	Running	0	...
istiod-xxxxxxxxxx-xxxxx	1/1	Running	0	...

Mensaje para explicar en clase:

Kubernetes despliega y orquesta contenedores. Istio controla como se comunican. Istio agrega seguridad, control de trafico y observabilidad sin obligarnos a reescribir la aplicacion.

7. Parte 3 - Activar sidecar injection

7.1 Etiquetar el namespace de la aplicacion

Si la aplicacion esta en el namespace `default`:

```
kubectl label namespace default istio-injection=enabled --overwrite
```

Validar etiqueta:

```
kubectl get namespace default --show-labels
```

Debe aparecer:

```
istio-injection=enabled
```

7.2 Reiniciar deployments

```
kubectl rollout restart deployment azure-vote-front  
kubectl rollout restart deployment azure-vote-back
```

Esperar a que terminen:

```
kubectl rollout status deployment azure-vote-front  
kubectl rollout status deployment azure-vote-back
```

7.3 Validar sidecars

```
kubectl get pods
```

Resultado esperado despues de Istio:

NAME	READY	STATUS	RESTARTS	AGE
azure-vote-back-xxxxxxxxx-xxxxx	2/2	Running	0	...
azure-vote-front-xxxxxxxxx-xxxxx	2/2	Running	0	...

Validar nombres de contenedores:

```
FRONT_POD=$(kubectl get pod -l app=azure-vote-front -o jsonpath='{.items[0].metadata.name}')  
kubectl get pod $FRONT_POD -o jsonpath='{.spec.containers[*].name}'  
echo
```

Resultado esperado:

```
azure-vote-front istio-proxy
```

Mensaje para explicar en clase:

El contenedor `istio-proxy` es Envoy. La aplicacion no fue modificada, pero ahora el trafico entra y sale por un proxy controlado por Istio. Esta es la base para aplicar politicas de seguridad desde la plataforma.

8. Parte 4 - Exponer la aplicacion con Istio Gateway

Ahora se publicara la aplicacion a traves de Istio Ingress Gateway.

8.1 Crear Gateway

```
cat <<EOF > gateway.yaml
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: microservices-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"
EOF
```

Aplicar:

```
kubectl apply -f gateway.yaml
```

8.2 Crear VirtualService

```
cat <<EOF > virtualservice.yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: azure-vote-vs
spec:
  hosts:
  - "*"
  gateways:
  - microservices-gateway
  http:
  - route:
    - destination:
        host: azure-vote-front
        port:
```



```
EOF      number: 80
```

Aplicar:

```
kubectl apply -f virtualservice.yaml
```

8.3 Obtener IP publica del Istio Gateway

```
kubectl get svc istio-ingressgateway -n istio-system
```

Guardar la IP:

```
export GATEWAY_IP=<EXTERNAL-IP-ISTIO>
```

Probar:

```
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP
```

Resultado esperado:

```
200
```

Mensaje para explicar en clase:

Ahora el trafico entra por Istio Gateway. Desde este punto podremos controlar autenticacion, autorizacion, trafico y observabilidad desde el mesh.

9. Parte 5 - Activar mTLS STRICT

mTLS permite cifrado y autenticacion mutua entre servicios. No solo protege la confidencialidad del trafico, tambien permite validar la identidad criptografica de los workloads.

9.1 Crear politica mTLS STRICT

```
cat <<EOF > mtls-strict.yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: default
spec:
  mtls:
    mode: STRICT
EOF
```

Aplicar:

```
kubectl apply -f mtls-strict.yaml
```

9.2 Validar estado de mTLS

```
istioctl authn tls-check
```

Si el comando no muestra suficiente detalle, validar con:

```
istioctl proxy-config cluster $FRONT_POD --fqdn azure-vote-back.default.svc.cluster.local
```

Mensaje para explicar en clase:

mTLS no es solo cifrado. Es autenticación mutua. El cliente sabe con quien habla y el servidor sabe quien lo está llamando. Esta es una pieza central de Zero Trust para tráfico east-west.

10. Parte 6 - Prueba negativa desde fuera del mesh

Ahora se demostrará que un pod sin sidecar no debería poder comunicarse correctamente con servicios que exigen mTLS STRICT.

10.1 Crear namespace sin Istio

```
kubectl create namespace outside
```

Validar que no tenga inyección:

```
kubectl get namespace outside --show-labels
```

10.2 Crear pod de prueba sin sidecar

```
kubectl run curl-outside \
  -n outside \
  --image=curlimages/curl \
  --restart=Never \
  --command -- sleep 3600
```

Validar:

```
kubectl get pod -n outside
```

10.3 Intentar llamada al servicio interno

```
kubectl exec -n outside curl-outside -- \
  curl -s -o /dev/null -w "%{http_code}\n" \
  http://azure-vote-front.default.svc.cluster.local
```

Resultado esperado:

```
000
```

Tambien podria verse un error de conexion o fallo TLS.

Mensaje para explicar en clase:

Antes, cualquier pod dentro del cluster podia intentar llamar al servicio. Con mTLS STRICT, solo workloads dentro del mesh y con identidad valida pueden comunicarse correctamente.

11. Parte 7 - Crear identidades de servicio

Para aplicar autorizacion granular, los servicios no deberian usar el service account `default`.

11.1 Crear service accounts

```
kubectl create serviceaccount azure-vote-front-sa
kubectl create serviceaccount azure-vote-back-sa
```

11.2 Asociar service accounts a deployments

```
kubectl patch deployment azure-vote-front \
  -p '{"spec":{"template":{"spec":{"serviceAccountName":"azure-vote-front-sa"}}}}'
```

```
kubectl patch deployment azure-vote-back \
  -p '{"spec":{"template":{"spec":{"serviceAccountName":"azure-vote-back-sa"}}}}'
```

Esperar rollout:

```
kubectl rollout status deployment azure-vote-front
kubectl rollout status deployment azure-vote-back
```

Actualizar variable del pod front:

```
FRONT_POD=$(kubectl get pod -l app=azure-vote-front -o jsonpath='{.items[0].metadata.name}')
```

Mensaje para explicar en clase:

En Istio la identidad del workload se basa en el service account de Kubernetes. Si todo usa default , no podemos aplicar reglas finas. Con service accounts dedicados podemos diferenciar identidades y aplicar microsegmentacion.

12. Parte 8 - AuthorizationPolicy: deny by default

La postura Zero Trust recomienda negar por defecto y permitir solo lo necesario.

12.1 Crear politica deny-all

```
cat <<EOF > deny-all.yaml
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-all
  namespace: default
spec: {}
EOF
```

Aplicar:

```
kubectl apply -f deny-all.yaml
```

12.2 Probar acceso desde Istio Gateway

```
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP
```

Resultado esperado:

```
403
```

Mensaje para explicar en clase:

Esta es una postura Zero Trust real: negar todo por defecto. Despues abrimos unicamente los flujos requeridos.

13. Parte 9 - Permitir trafico desde Istio Gateway hacia front

13.1 Identificar service account del ingress gateway

```
kubectl get sa -n istio-system
```

En una instalacion estandar con `istioctl` , normalmente sera:

```
istio-ingressgateway-service-account
```

El principal esperado sera:

```
cluster.local/ns/istio-system/sa/istio-ingressgateway-service-account
```

Nota: si se usa el add-on administrado de AKS, el namespace y el service account pueden ser diferentes. En ese caso, ajustar el principal segun el service account real del ingress gateway.

13.2 Crear politica allow desde gateway hacia front

```
cat <<EOF > allow-ingress-to-front.yaml
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-ingress-to-front
  namespace: default
spec:
  selector:
    matchLabels:
      app: azure-vote-front
  action: ALLOW
  rules:
  - from:
    - source:
        principals:
        - "cluster.local/ns/istio-system/sa/istio-ingressgateway-service-account"
EOF
```

Aplicar:

```
kubectl apply -f allow-ingress-to-front.yaml
```

13.3 Crear politica allow desde front hacia back

```
cat <<EOF > allow-front-to-back.yaml
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-front-to-back
  namespace: default
spec:
  selector:
    matchLabels:
      app: azure-vote-back
  action: ALLOW
  rules:
  - from:
    - source:
        principals:
        - "cluster.local/ns/default/sa/azure-vote-front-sa"
EOF
```

Aplicar:

```
kubectl apply -f allow-front-to-back.yaml
```

13.4 Probar acceso nuevamente

```
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP
```

Resultado esperado:

```
200
```

Mensaje para explicar en clase:

No abrimos todo el namespace. Solo permitimos dos flujos: gateway hacia front y front hacia backend. Esto es microsegmentacion logica basada en identidad de workload.

14. Parte 10 - Validar JWT con Istio

Ahora se agregara autenticacion basada en token. Istio validara el JWT antes de permitir acceso al servicio.

Para simplificar el laboratorio, se generara un JWT local con una clave RSA y se insertara el JWKS directamente en la politica. Asi no dependemos de Keycloak ni de un proveedor externo durante la clase.

14.1 Instalar dependencias Python

```
python3 -m pip install --user pyjwt cryptography jwcrypto
```

Si Cloud Shell muestra advertencias de PATH, normalmente no afectan este laboratorio.

14.2 Crear script para generar JWT y JWKS

```
cat <<'EOF' > generate-jwt.py
from jwcrypto import jwk
import jwt
import time
import json

ISSUER = "lab@istio.local"
KID = "lab-key-1"

key = jwk.JWK.generate(kty="RSA", size=2048, kid=KID)

private_pem = key.export_to_pem(private_key=True, password=None)
public_jwk = json.loads(key.export(private_key=False))
public_jwk["kid"] = KID

jwks = {"keys": [public_jwk]}

now = int(time.time())
```

```

valid_payload = {
    "iss": ISSUER,
    "sub": "student-user",
    "aud": "azure-vote",
    "iat": now,
    "exp": now + 3600,
    "scope": "vote:read",
    "role": "student"
}

invalid_payload = {
    "iss": ISSUER,
    "sub": "guest-user",
    "aud": "azure-vote",
    "iat": now,
    "exp": now + 3600,
    "scope": "wrong:scope",
    "role": "guest"
}

valid_token = jwt.encode(
    valid_payload,
    private_pem,
    algorithm="RS256",
    headers={"kid": KID}
)

invalid_token = jwt.encode(
    invalid_payload,
    private_pem,
    algorithm="RS256",
    headers={"kid": KID}
)

with open("jwks.json", "w") as f:
    json.dump(jwks, f)

with open("valid-token.txt", "w") as f:
    f.write(valid_token)

with open("invalid-scope-token.txt", "w") as f:
    f.write(invalid_token)

print("Generated files:")
print("- jwks.json")
print("- valid-token.txt")
print("- invalid-scope-token.txt")
EOF

```

Ejecutar:

```
python3 generate-jwt.py
```

Validar archivos:

```
ls -l jwks.json valid-token.txt invalid-scope-token.txt
```

14.3 Revisar token valido

```
cat valid-token.txt
```

Opcional: copiar el token en jwt.io para revisar sus claims.

Claims esperados:

```
{
  "iss": "lab@istio.local",
  "sub": "student-user",
  "aud": "azure-vote",
  "scope": "vote:read",
  "role": "student"
}
```

Mensaje para explicar en clase:

El payload del JWT no esta cifrado. Cualquiera puede decodificarlo. La seguridad no viene de ocultar el payload, sino de verificar la firma, la expiracion, el issuer, la audience y los claims relevantes.

15. Parte 11 - Crear RequestAuthentication con JWKS embebido

15.1 Preparar JWKS para YAML

```
JWKS=$(cat jwks.json | sed 's/" /\\"/g')
```

15.2 Crear RequestAuthentication

```
cat <<EOF > request-authentication.yaml
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: azure-vote-jwt
  namespace: default
spec:
  selector:
    matchLabels:
      app: azure-vote-front
  jwtRules:
    - issuer: "lab@istio.local"
      audiences:
        - "azure-vote"
      jwks: "$JWKS"
EOF
```

Aplicar:

```
kubectl apply -f request-authentication.yaml
```

Validar:

```
kubectl get requestauthentication
```

Mensaje para explicar en clase:

RequestAuthentication le dice a Istio como validar el JWT: quien es el issuer, cual es la audience esperada y que clave publica debe usar para verificar la firma.

16. Parte 12 - Exigir JWT valido y scope correcto

La politica anterior permitia gateway hacia front sin exigir JWT. Ahora se reemplazara por una politica mas estricta.

16.1 Eliminar politica anterior

```
kubectl delete authorizationpolicy allow-ingress-to-front
```

16.2 Crear politica que exige JWT y claim `scope=vote:read`

```
cat <<EOF > allow-ingress-to-front-with-jwt.yaml
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-ingress-to-front-with-jwt
  namespace: default
spec:
  selector:
    matchLabels:
      app: azure-vote-front
  action: ALLOW
  rules:
    - from:
      - source:
          principals:
            - "cluster.local/ns/istio-system/sa/istio-ingressgateway-service-account"
          requestPrincipals:
            - "lab@istio.local/*"
        when:
          - key: request.auth.claims[scope]
            values:
              - "vote:read"
EOF
```

Aplicar:

```
kubectl apply -f allow-ingress-to-front-with-jwt.yaml
```

Mensaje para explicar en clase:

Ya no basta con entrar por el gateway. Ahora el request debe traer un JWT valido, emitido por el issuer esperado, con la audience correcta y con el scope requerido.

17. Parte 13 - Pruebas de seguridad

17.1 Prueba 1: sin token

```
curl -s -o /dev/null -w "%{http_code}\n" http://$GATEWAY_IP
```

Resultado esperado:

403

Explicacion:

No hay JWT.
No hay requestPrincipal.
Istio rechaza el request.

17.2 Prueba 2: token valido en firma, pero con scope incorrecto

```
BAD_TOKEN=$(cat invalid-scope-token.txt)
```

```
curl -s -o /dev/null -w "%{http_code}\n" \
-H "Authorization: Bearer $BAD_TOKEN" \
http://$GATEWAY_IP
```

Resultado esperado:

403

Explicacion:

El JWT esta firmado correctamente.
El issuer es valido.
La audience es valida.
Pero el scope no es vote:read.
Istio rechaza el request.

17.3 Prueba 3: token valido con scope correcto

```
VALID_TOKEN=$(cat valid-token.txt)
```

```
curl -s -o /dev/null -w "%{http_code}\n" \
-H "Authorization: Bearer $VALID_TOKEN" \
http://$GATEWAY_IP
```

Resultado esperado:

200

17.4 Prueba 4: token alterado manualmente

```
TAMPERED_TOKEN="${VALID_TOKEN}abc"
```

```
curl -s -o /dev/null -w "%{http_code}\n" \
-H "Authorization: Bearer $TAMPERED_TOKEN" \
http://$GATEWAY_IP
```

Resultado esperado:

```
401
```

O bien:

```
403
```

Dependera de como Istio reporte el fallo de autentificacion/autorizacion.

Mensaje para explicar en clase:

Ya tenemos una cadena de seguridad completa: entrada por Istio Gateway, JWT validado por Istio, autorizacion basada en claims, mTLS interno y autorizacion service-to-service.

18. Parte 14 - Validaciones finales

18.1 Ver politicas de autentificacion

```
kubectl get requestauthentication
```

Resultado esperado:

NAME	AGE
azure-vote-jwt	...

18.2 Ver politicas de autorizacion

```
kubectl get authorizationpolicy
```

Resultado esperado:

NAME	AGE
deny-all	...
allow-front-to-back	...
allow-ingress-to-front-with-jwt	...

18.3 Ver politicas mTLS

```
kubectl get peerauthentication
```

Resultado esperado:

NAME	MODE	AGE
default	STRICT	...

18.4 Analizar configuracion de Istio

```
istioctl analyze
```

Resultado esperado:

```
No validation issues found.
```

18.5 Revisar configuracion distribuida al proxy

```
FRONT_POD=$(kubectl get pod -l app=azure-vote-front -o jsonpath='{.items[0].metadata.name}')
```

Listeners:

```
istioctl proxy-config listeners $FRONT_POD
```

Clusters:

```
istioctl proxy-config clusters $FRONT_POD
```

Secrets/certificados:

```
istioctl proxy-config secret $FRONT_POD
```

Mensaje para explicar en clase:

En produccion no basta con aplicar YAML. Debemos validar que las politicas realmente llegaron al proxy Envoy y que el comportamiento observado coincide con la intencion de seguridad.

19. Parte 15 - Integracion opcional con Jenkins

Esta parte puede mostrarse como demo final o dejarse como ejercicio.

La idea es versionar los manifiestos de Istio dentro del repositorio:

```
repo/  
azure-vote/
```

```
k8s/
istio/
  gateway.yaml
  virtualservice.yaml
  mtls-strict.yaml
  deny-all.yaml
  allow-front-to-back.yaml
  request-authentication.yaml
  allow-ingress-to-front-with-jwt.yaml
```

En Jenkins, despues del despliegue de la imagen:

```
kubectl apply -f istio/gateway.yaml
kubectl apply -f istio/virtualservice.yaml
kubectl apply -f istio/mtls-strict.yaml
kubectl apply -f istio/deny-all.yaml
kubectl apply -f istio/allow-front-to-back.yaml
kubectl apply -f istio/request-authentication.yaml
kubectl apply -f istio/allow-ingress-to-front-with-jwt.yaml
```

19.1 Smoke test sin token

```
STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://$GATEWAY_IP)

if [ "$STATUS" != "403" ]; then
  echo "Security test failed: endpoint should reject unauthenticated requests"
  exit 1
fi
```

19.2 Smoke test con token valido

```
VALID_TOKEN=$(cat valid-token.txt)

STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
  -H "Authorization: Bearer $VALID_TOKEN" \
  http://$GATEWAY_IP)

if [ "$STATUS" != "200" ]; then
  echo "Security test failed: valid token should be accepted"
  exit 1
fi
```

Mensaje para explicar en clase:

Ahora la seguridad no se aplica manualmente. Se versiona, se despliega y se prueba dentro del pipeline. Esto es Security as Code aplicado a microservicios.

20. Resultado esperado final

Al finalizar, la arquitectura queda asi:

```
Cliente externo
-> HTTP request con JWT
-> Istio Ingress Gateway
```

```
-> RequestAuthentication valida firma, issuer y audience
-> AuthorizationPolicy valida principal y scope
-> azure-vote-front
-> mTLS STRICT hacia azure-vote-back
-> AuthorizationPolicy permite solo front -> back
```

Pruebas esperadas:

Prueba	Resultado esperado
Sin token	403
Token alterado	401 o 403
Token con scope incorrecto	403
Token valido con <code>scope=vote:read</code>	200
Pod fuera del mesh llamando servicio protegido	Falla o codigo 000
Front llamando a backend	Permitido

21. Troubleshooting

Problema 1: Los pods siguen mostrando 1/1

Posible causa: el namespace no tiene inyeccion habilitada o los pods no fueron reiniciados.

Solucion:

```
kubectl get namespace default --show-labels
kubectl label namespace default istio-injection=enabled --overwrite
kubectl rollout restart deployment azure-vote-front
kubectl rollout restart deployment azure-vote-back
```

Problema 2: El Gateway no responde

Validar servicio de ingreso:

```
kubectl get svc istio-ingressgateway -n istio-system
```

Validar Gateway y VirtualService:

```
kubectl get gateway
kubectl get virtualservice
istioctl analyze
```

Problema 3: Todo responde 403 despues de deny-all

Es esperado hasta crear las politicas ALLOW.

Validar:

```
kubectl get authorizationpolicy
kubectl describe authorizationpolicy allow-ingress-to-front-with-jwt
```

Problema 4: El principal del ingress gateway no coincide

Validar service accounts:

```
kubectl get sa -n istio-system
```

Si el nombre cambia, modificar el principal en la AuthorizationPolicy:

```
cluster.local/ns/<namespace>/sa/<service-account>
```

Problema 5: El JWT siempre falla

Validar que el token no haya expirado:

```
python3 generate-jwt.py
VALID_TOKEN=$(cat valid-token.txt)
```

Validar que los valores coincidan:

```
issuer: lab@istio.local
audience: azure-vote
scope: vote:read
```

Problema 6: Istio no reconoce el JWKS embebido

Recrear el manifiesto:

```
JWKS=$(cat jwks.json | sed 's/" /\\"/g')
```

Luego recrear request-authentication.yaml y aplicar:

```
kubectl apply -f request-authentication.yaml
```

22. Limpieza del laboratorio

Eliminar políticas:

```
kubectl delete authorizationpolicy allow-ingress-to-front-with-jwt --ignore-not-found
kubectl delete authorizationpolicy allow-front-to-back --ignore-not-found
kubectl delete authorizationpolicy deny-all --ignore-not-found
kubectl delete requestauthentication azure-vote-jwt --ignore-not-found
kubectl delete peerauthentication default --ignore-not-found
kubectl delete virtualservice azure-vote-vs --ignore-not-found
kubectl delete gateway microservices-gateway --ignore-not-found
kubectl delete namespace outside --ignore-not-found
```

Quitar etiqueta de inyeccion:

```
kubectl label namespace default istio-injection- --overwrite
```

Opcional: desinstalar Istio por completo:

```
istioctl uninstall --purge -y  
kubectl delete namespace istio-system --ignore-not-found
```

Reiniciar deployments para quitar sidecars:

```
kubectl rollout restart deployment azure-vote-front  
kubectl rollout restart deployment azure-vote-back
```

23. Preguntas de cierre para los alumnos

1. Que problema resuelve mTLS que no resuelve solamente HTTPS en el ingreso?
 2. Por que no deberiamos usar el service account `default` para todos los microservicios?
 3. Que diferencia hay entre autentificacion JWT y autorizacion por claims?
 4. Que ventaja tiene aplicar AuthorizationPolicy fuera del codigo de la aplicacion?
 5. Como integrarian estas politicas en un pipeline CI/CD real?
 6. Que riesgos quedan sin resolver aunque usemos Istio?
-

24. Mensaje final del laboratorio

La seguridad en microservicios no debe depender de una sola capa. En este laboratorio se implemento un enfoque de defensa en profundidad:

- El gateway controla la entrada.
- JWT valida identidad de usuario o cliente.
- Los claims limitan lo que se puede hacer.
- mTLS protege comunicacion interna.
- AuthorizationPolicy controla quien puede hablar con quien.
- Jenkins puede desplegar estas politicas como codigo.

La idea principal es clara: en una arquitectura cloud-native moderna, la seguridad debe ser declarativa, automatizable, observable y alineada con Zero Trust.

25. Referencias recomendadas

- Istio Documentation - Security: <https://istio.io/latest/docs/concepts/security/>
- Istio Task - Mutual TLS Migration: <https://istio.io/latest/docs/tasks/security/authentication/mtls-migration/>

- Istio Task - Authorization with JWT: <https://istio.io/latest/docs/tasks/security/authorization/authz-jwt/>
- Istio Reference - AuthorizationPolicy: <https://istio.io/latest/docs/reference/config/security/authorization-policy/>
- Microsoft Learn - Istio-based service mesh add-on for AKS: <https://learn.microsoft.com/azure/aks/istio-about>
- OWASP API Security Top 10 2023: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>